

NoxVault

Documentação Técnica Completa

Cofre de Senhas Local · Biometria Android · Análise de Segurança por IA

Versão 1.0.0 · 07/06/2026 · Flutter 3.x / Dart 3

Plataforma	Criptografia	Hash PIN	IA	Armazenamento
Android	AES-256-GCM	PBKDF2	GROO LLaMA	Local

1. Visão Geral

O NoxVault é um gerenciador de senhas de código aberto desenvolvido em Flutter, voltado para o ecossistema Android. Seu diferencial é a combinação de três pilares: **armazenamento 100% local** (nenhum dado sai do dispositivo sem consentimento do usuário), **criptografia forte** (AES-256-GCM com derivação PBKDF2-SHA256) e **inteligência artificial** para análise de qualidade das senhas via GROQ API, transmitindo apenas metadados — jamais as senhas em texto puro.

1.1 Objetivos do Projeto

- Proteger credenciais sensíveis com criptografia de nível militar, sem dependência de servidores externos.
- Oferecer experiência de autenticação fluida combinando biometria facial/digital e PIN.
- Empoderar o usuário com análise inteligente da saúde do seu cofre de senhas.
- Garantir total transparência: código auditável, sem coleta de dados, sem telemetria.

1.2 Stack Tecnológica

Pacote / Tecnologia	Versão	Finalidade
flutter + provider	SDK 3.4+ / 6.1.2	Framework UI e gerenciamento de estado
local_auth	^3.0.1	Biometria nativa Android (FlutterFragmentActivity)
flutter_secure_storage	^10.0.0	Keystore / Keychain do sistema operacional
encrypt	^5.0.3	Cifração AES-256-GCM das notas
pointycastle	^3.9.1	PBKDF2-SHA256 para derivação de chave
crypto	^3.0.3	SHA-256 utilitário
http	^1.2.2	Chamadas REST à API GROQ
connectivity_plus	^7.1.1	Verificação de conectividade de rede

2. Arquitetura do Projeto

O projeto segue o padrão **Provider + Service Layer**: a UI (screens/widgets) consome Providers que, por sua vez, delegam operações a serviços especializados. Nenhuma lógica de negócio reside nas telas.

2.1 Estrutura de Diretórios

Estrutura de diretórios – lib/

```
lib/
  ■■■ main.dart                ← Ponto de entrada, setup do Provider
  ■■■ providers/
    ■ ■■■ auth_provider.dart    ← Autenticação: PIN, biometria, logout
    ■ ■■■ vault_provider.dart    ← CRUD de notas, re-cifragem
  ■■■ services/
    ■ ■■■ ai_security_service.dart ← GROQ API / LLaMA
    ■ ■■■ crypto_service.dart     ← AES-256-GCM + PBKDF2-SHA256
    ■ ■■■ secure_storage_service.dart ← Wrapper Flutter Secure Storage
  ■■■ screens/
    ■ ■■■ login_screen.dart
    ■ ■■■ home_screen.dart
    ■ ■■■ security_analysis_screen.dart
    ■ ■■■ change_master_pin_screen.dart
  ■■■ models/
    ■ ■■■ note.dart
    ■ ■■■ security_analysis.dart
  ■■■ widgets/
    ■ ■■■ note_form_sheet.dart
    ■ ■■■ password_hint_row.dart
    ■ ■■■ forgot_pin_dialog.dart
  ■■■ utils/
    ■ ■■■ password_security.dart
```

2.2 Fluxo de Dados

Camada	Responsabilidade	Arquivo Principal
UI / Screens	Renderização, eventos do usuário	login_screen.dart, home_screen.dart
Providers	Estado global, orquestração de casos de uso	auth_provider.dart, vault_provider.dart
Services	Lógica pura: cripto, IA, storage	crypto_service.dart, ai_security_service.dart
Models	Entidades de domínio (Note, SecurityAnalysis)	models/note.dart
Utils	Funções auxiliares (força de senha, gerador)	password_security.dart

3. Sistema de Autenticação

3.1 Criação do PIN

No primeiro acesso, o usuário cria um PIN numérico de 4 a 6 dígitos. O PIN nunca é armazenado em texto puro — o fluxo de persistência é:

- **1. Geração do salt:** 32 bytes aleatórios via `SecureRandom` (Pointycastle).
- **2. Hash PBKDF2-SHA256:** 200.000 iterações, saída de 32 bytes.
- **3. Armazenamento:** formato `base64(salt):base64(hash)` gravado no Android Keystore via `flutter_secure_storage`.
- **4. Derivação da chave AES:** a mesma rotina PBKDF2 deriva 32 bytes usados como chave AES-256.
- **5. Persistência da chave AES:** chave salva em base64 no Keystore para uso no desbloqueio biométrico.

3.2 Desbloqueio por PIN

Etapa	Ação	Arquivo / Linha
1. Leitura do hash	<code>readPinHash()</code> lê <code>salt:hash</code> do Keystore	<code>secure_storage_service.dart</code>
2. Verificação	<code>verifyPin()</code> recalcula PBKDF2 e compara em tempo constante	<code>crypto_service.dart</code>
3. Derivação da chave	<code>deriveKey()</code> gera a chave AES-256 em memória	<code>auth_provider.dart</code>
4. Persistência	<code>writeVaultKey()</code> salva chave base64 no Keystore	<code>auth_provider.dart</code>
5. Autenticado	<code>_isAuthenticated = true;</code> <code>_vaultKey</code> em memória	<code>auth_provider.dart</code>

3.3 Desbloqueio por Biometria

A biometria não deriva a chave AES diretamente — ela atua como portão de acesso à chave previamente armazenada no Keystore. Isso evita a necessidade de re-digitar o PIN a cada abertura do app, mantendo a segurança intacta.

■ Requisito Android: `MainActivity` deve estender `FlutterFragmentActivity` (não `FlutterActivity`) para que o diálogo biométrico do sistema seja exibido corretamente.

- **1.** `_localAuth.authenticate()` exibe o diálogo nativo (face ou digital conforme configurado no aparelho).
- **2.** Em caso de sucesso: `readVaultKey()` lê a chave base64 do Keystore.
- **3.** `base64.decode()` restaura a `Uint8List` em memória como `_vaultKey`.
- **4.** `_isAuthenticated = true` — o cofre é aberto sem solicitar o PIN.
- **5.** Após 2 falhas biométricas: fallback automático para formulário de PIN.

3.4 Bloqueio Progressivo de PIN

Tentativas Erradas	Comportamento
1ª e 2ª	Mensagem de erro com contagem restante

Tentativas Erradas	Comportamento
3ª	Bloqueio de 1 minuto; contador zerado após o prazo
Durante bloqueio	Campo de PIN bloqueado; timer regressivo exibido na tela

3.5 Recuperação de PIN ("Esqueci o PIN")

Quando o usuário esquece o PIN, o fluxo exige autenticação biométrica para autorizar a criação de um novo PIN. Por questão de integridade criptográfica, as notas cifradas com a chave antiga são apagadas — não há como decifrá-las sem o PIN original.

- Comportamento esperado: ao redefinir o PIN, o cofre é limpo. Este é um trade-off deliberado de segurança: sem o PIN original, a chave AES não pode ser recuperada e as notas não podem ser decifradas.

A sessão de recuperação tem validade de **3 minutos** (`kPinRecoverySessionTtl`). Após biometria validada, o usuário define o novo PIN e confirma. O estado `pinRecoveryArmed` é apagado imediatamente após a conclusão.

4. Cofre de Senhas

4.1 Modelo de Dados — Note

Campo	Tipo	Descrição
id	String (UUID)	Identificador único gerado automaticamente
title	String	Nome do site/serviço
username	String	Login / e-mail
password	String	Senha (cifrada em repouso com AES-256-GCM)
notes	String	Anotações livres (cifradas junto com a senha)
createdAt	DateTime	Data de criação da entrada
updatedAt	DateTime	Data da última modificação (usado na análise de idade)

4.2 Criptografia AES-256-GCM

Cada nota é cifrada individualmente usando AES no modo GCM (Galois/Counter Mode), que oferece **confidencialidade + autenticidade** — qualquer adulteração no ciphertext é detectada na decifração.

Implementação de cifragem

```
// Cifragem (crypto_service.dart)
String encrypt(String plaintext, Uint8List key) {
  final iv = IV.fromSecureRandom(16); // 16 bytes aleatórios
  final encrypter = Encrypter(AES(Key(key), mode: AESMode.gcm));
  final encrypted = encrypter.encrypt(plaintext, iv: iv);
  return '${base64.encode(iv.bytes)}:${encrypted.base64}';
}

// Formato de armazenamento: base64(iv):base64(ciphertext+tag)
```

4.3 Funcionalidades do Cofre

Feature	Descrição	Segurança
Criar entrada	Formulário com título, usuário, senha e notas	Cifrado antes de persistir
Editar entrada	Sheet bottom com campos pré-preenchidos	Re-cifrado na nova chave
Excluir entrada	Confirmação via dialog antes de remover	Dados apagados do Keystore
Reveal de senha	Exibir senha de uma entrada específica	Requer biometria
Copiar senha	Copia para clipboard com um toque	Clipboard limpo em 30s
Busca	Filtra por título ou usuário em tempo real	Dados em memória cifrada
Gerador de senhas	Configura comprimento, maiúsculas, números e símbolos	Entropia via SecureRandom
Indicador de força	5 níveis: Muito Fraca → Muito Forte	Feedback imediato

Feature	Descrição	Segurança
Alterar PIN	Exige PIN atual ou biometria	Re-cifra todas as notas
Logout	Apaga _vaultKey da memória imediatamente	Zero dados sensíveis em RAM

5. Análise de Segurança com IA

5.1 Visão Geral da Integração GROQ

A análise de segurança utiliza o modelo **LLaMA 3.3 70B Versatile** via GROQ API (endpoint OpenAI-compatível). O modelo recebe apenas metadados anonimizados das senhas — nenhuma senha em texto puro é transmitida. A resposta é um JSON estruturado com score, alertas, sugestões e pontos positivos.

■ Privacidade: apenas metadados são enviados à API — comprimento, tipos de caractere (booleanos), flag de reutilização e idade em dias. O nome do site é enviado para contextualizar os alertas, mas nenhuma senha viaja pela rede.

5.2 Metadados Transmítidos

Payload JSON enviado à GROQ API (metadados apenas)

```
// Por entrada (ai_security_service.dart)
{
  'site':           note.title,           // nome do serviço
  'passwordLength': pwd.length,           // comprimento numérico
  'hasUppercase':   bool,                 // contém A-Z
  'hasLowercase':   bool,                 // contém a-z
  'hasNumbers':     bool,                 // contém 0-9
  'hasSpecialChars': bool,               // contém !@#$%...
  'isReused':       bool,                 // igual a outra senha
  'ageDays':        int,                  // dias desde atualização
  'startsWithCommonWord': bool          // 'senha', 'pass', '123'...
}
```

5.3 Resposta da IA — SecurityAnalysis

Campo	Tipo	Descrição
score	int (0–100)	Pontuação geral de segurança do cofre
nivel	String	Crítico Fraco Médio Bom Excelente
totalPasswords	int	Total de entradas analisadas
reusedCount	int	Quantidade de senhas reutilizadas
weakCount	int	Quantidade de senhas fracas
oldCount	int	Senhas com mais de 180 dias sem atualização
alertas	List<String>	Até 5 problemas críticos identificados
sugestoes	List<String>	Até 5 recomendações práticas
destaquesPositivos	List<String>	Até 3 pontos fortes encontrados
resumo	String	Resumo em até 20 palavras

5.4 Gerenciamento da Chave GROQ

Prioridade	Fonte	Como configurar
1ª (compilação)	Dart define env	<code>flutter run --dart-define=GROQ_API_KEY=gsk_...</code>
2ª (persistida)	Android Keystore	Inserida uma vez pelo usuário via dialog
3ª (manual)	Dialog em tela	Solicitada a cada uso (não salva)

A chave é validada para o prefixo `gsk_` e comprimento mínimo de 20 caracteres antes de ser aceita. Após digitada, é armazenada no Keystore com a chave `noxvault_groq_api_key`. O usuário pode redefinir a chave pelo menu da tela de análise a qualquer momento.

5.5 Tratamento de Erros da API

Tipo de Erro	Condição	Mensagem ao Usuário
<code>noInternet</code>	Sem conectividade detectada	Verifique sua rede e tente novamente
<code>invalidApiKey</code>	HTTP 401/403 da API	Chave inválida — redefina pelo menu
<code>timeout</code>	> 30 segundos sem resposta	Análise demorou muito, tente novamente
<code>invalidResponse</code>	JSON malformado na resposta	Resposta inesperada da IA
<code>noPasswordsFound</code>	Nenhuma nota com senha no cofre	Adicione entradas com senha primeiro
<code>unknown</code>	Qualquer outra exception	Mensagem de erro técnico exibida

6. Modelo de Segurança

6.1 Camadas de Proteção

Camada	Mecanismo	Parâmetros
Hash do PIN	PBKDF2-SHA256	Salt 32 bytes · 200.000 iterações · 32 bytes de saída
Chave AES	PBKDF2 derivado do PIN	256 bits · armazenado no Android Keystore
Notas	AES-256-GCM	IV aleatório 16 bytes por operação
Biometria	API nativa Android	Face / Digital via FlutterFragmentActivity
Keystore	flutter_secure_storage	EncryptedSharedPreferences no Android
Clipboard	Timer de limpeza	30 segundos após cópia
Lockout de PIN	Contador persistido	3 erros → 1 min de bloqueio
Sessão biométrica	Chave em memória + Keystore	Apagada do RAM no logout

6.2 O que NÃO é armazenado

- ✗ O PIN nunca é armazenado — apenas seu hash PBKDF2 com salt.
- ✗ A chave AES em memória (`_vaultKey`) nunca é persistida em texto puro.
- ✗ Nenhuma senha é enviada a servidores externos em texto puro.
- ✗ A chave GROQ API é salva exclusivamente no Keystore do Android, nunca em SharedPreferences comum.
- ✗ Nenhuma telemetria, analytics ou log remoto é coletado.

6.3 Permissões Android Utilizadas

AndroidManifest.xml



A permissão `INTERNET` é requerida exclusivamente para chamadas à GROQ API. O cofre de senhas funciona 100% offline — apenas a análise de IA requer conexão.

7. Telas e Fluxo de Usuário

7.1 login_screen.dart

Tela inicial do app. Gerencia os três estados possíveis de acesso: criação de PIN (primeiro uso), fase biométrica e fase de PIN. O estado é derivado de `isBiometricPhase` e `shouldOfferPinEntry` no `AuthProvider`.

Estado	Condição	UI Exibida
Primeiro acesso	<code>!hasStoredPin</code>	Campo de PIN + botão 'Criar PIN e entrar'
Fase biométrica	<code>hasStoredPin && biometricAvailable && tentativas < 2</code>	Botão de biometria em destaque
Fase PIN	<code>shouldOfferPinEntry == true</code>	Campo de PIN + 'Esqueci o PIN'
Bloqueado	<code>isPinLockedOut == true</code>	Timer regressivo + campo desabilitado

7.2 home_screen.dart

Tela principal do cofre após autenticação. Exibe a lista de entradas com busca, botão de adicionar, acesso à análise de IA e menu de opções (alterar PIN, logout). Cada card de nota oferece copiar usuário/senha e abrir o formulário de edição.

7.3 security_analysis_screen.dart

Tela de análise de segurança por IA. Apresenta três estados: initial (botão de análise), loading (spinner) e result (score, estatísticas, alertas, sugestões e destaques). Gerencia o ciclo de vida da chave GROQ (ler do Keystore → solicitar ao usuário → persistir no Keystore).

■ Correção implementada: `context.read()` é capturado ANTES de qualquer `await` para evitar o erro `'dependents.isEmpty is not true'` do Flutter.

7.4 change_master_pin_screen.dart

Permite alterar o PIN mestre de dentro do cofre. Oferece dois modos: confirmação via PIN atual (para quem lembra) ou confirmação via biometria. Ao concluir, todas as notas são re-cifradas automaticamente com a nova chave AES.

7.5 forgot_pin_dialog.dart (Widget)

Dialog modal acessível pela tela de login. Inicia o fluxo de recuperação de PIN: 1) dispara biometria via `startPinRecoveryWithBiometric()`, 2) se aprovado, exibe campos de novo PIN e confirmação, 3) chama `completeMasterPinRecovery()`. Sessão expira em 3 minutos.

8. Correções Implementadas

8.1 Biometria Não Funcionava (FlutterFragmentActivity)

Item	Detalhe
Arquivo	android/app/src/main/kotlin/com/example/noxvault/MainActivity.kt
Problema	MainActivity estendia FlutterActivity; o local_auth requer FlutterFragmentActivity para exibir o diálogo biométrico
Solução	Alterado: class MainActivity : FlutterFragmentActivity()
Impacto	Biometria facial e digital passaram a funcionar em todos os dispositivos Android

8.2 Biometria Falhava Silenciosamente (Vault Key Nula)

Item	Detalhe
Arquivo	lib/providers/auth_provider.dart + lib/services/secure_storage_service.dart
Problema	_vaultKey era mantida apenas em memória RAM. Ao fechar/reabrir o app, a chave sumia e a biometria retornava erro
Solução	Após PIN verificado: writeVaultKey() salva base64 no Keystore. Após biometria aprovada: readVaultKey() restaura
Segurança	A chave fica protegida pelo Android Keystore (EncryptedSharedPreferences), equivalente à proteção das notas

8.3 Crash ao Inserir Chave GROQ (_dependents.isEmpty)

Item	Detalhe
Arquivo	lib/screens/security_analysis_screen.dart — método _analyze()
Problema	context.read<VaultProvider>() era chamado após await _getOrRequestApiKey(). Em Flutter, usar context após await causa crash
Solução	notes capturado antes do primeiro await: final notes = context.read<VaultProvider>().notes (linha 1 do método)
Impacto	Análise de IA passou a funcionar sem crash para qualquer chave válida

8.4 Migração Gemini → GROQ API

Item	Detalhe
Arquivos	pubspec.yaml, ai_security_service.dart, security_analysis_screen.dart, AndroidManifest.xml
Problema	App usava google_generative_ai (Gemini) com chave no formato AlzaSy..., mas a chave disponível era GROQ
Solução	Substituído google_generative_ai por http ^1.2.2; endpoint: api.groq.com/openai/v1/chat/completions; modelo: llama3
Validação	Prefixo aceito alterado de 'AlzaSy' para 'gsk_'; textos da UI atualizados; permissão INTERNET adicionada ao M

9. Como Executar o Projeto

9.1 Pré-requisitos

- Flutter SDK >= 3.4.0 (dart >= 3.0.0)
- Android Studio com Android SDK (minSdk 21 / API Level 21+)
- Dispositivo ou emulador Android com biometria cadastrada (digital ou facial)
- Chave GROQ API obtida em <https://console.groq.com> (formato gsk_...)

9.2 Instalação e Execução

Comandos de build

```
# Instalar dependências
flutter pub get

# Executar no dispositivo/emulador conectado
flutter run

# Build APK de debug
flutter build apk --debug

# Build APK de release
flutter build apk --release
```

9.3 Inicializar o Emulador Android

Emulador Android

```
# Listar emuladores disponíveis
flutter emulators

# Iniciar emulador
flutter emulators --launch Medium_Phone

# Após emulador abrir, executar o app
flutter run
```

■ Biometria no emulador: configure uma digital simulada em Configurações → Segurança → Impressão digital. Para simular durante o desbloqueio, use Extended Controls → Fingerprint no emulador.

10. Referência Rápida — Constantes e Configurações

Constante	Valor	Descrição
kMaxBiometricAttemptsBeforePin	2	Tentativas biométricas antes de exigir PIN
kMaxPinFailuresBeforeLockout	3	Erros de PIN antes do bloqueio
kPinLockoutDuration	1 minuto	Duração do bloqueio por PIN
kPinRecoverySessionTtl	3 minutos	Validade da sessão de recuperação de PIN
PBKDF2 iterações	200.000	Custo computacional do hash do PIN
Salt size	32 bytes	Tamanho do salt aleatório do PIN
AES key size	256 bits	Comprimento da chave de cifragem
AES IV size	16 bytes	IV aleatório por operação de cifragem
Clipboard TTL	30 segundos	Tempo para limpeza automática do clipboard
GROQ model	llama-3.3-70b-versatile	Modelo LLM usado na análise
GROQ API timeout	30 segundos	Tempo máximo de espera da API
Senha 'antiga'	> 180 dias	Critério para flag de senha desatualizada
PIN mínimo	4 dígitos	Comprimento mínimo do PIN
PIN máximo	6 dígitos	Comprimento máximo do PIN

11. Considerações Finais

O NoxVault demonstra que é possível construir um gerenciador de senhas seguro, moderno e com IA sem abrir mão da privacidade do usuário. Todo o armazenamento é local, toda a criptografia é padrão da indústria, e a análise de IA é não-invasiva — operando apenas sobre metadados.

O projeto está preparado para evolução: a chave AES pode ser migrada para derivação via Keystore de hardware (StrongBox), a análise pode ser executada localmente com modelos on-device, e o suporte iOS pode ser ativado com ajustes mínimos de configuração.

■ Desenvolvido com Flutter 3.x · Dart 3 · local_auth 3.0.1 · GROQ LLaMA 3.3 70B